



TRƯỜNG ĐIỆN – ĐIỆN TỬ

KHOA ĐIỆN TỬ

KỸ THUẬT PHẦN MỀM ỨNG DỤNG

Chương 2

Các pha trong phát triển phần mềm

Phần 8

Thiết kế hệ thống – Thiết kế kiến trúc

3898043	5660163	63758	6752245	7196671	154963	2867239	17
OAS	ODS	MAV	WAV	IDS	VUIL	KAS	1 BAI



Kiến trúc phần mềm

- Khái niệm:

“Là một cấu trúc bao gồm các thành phần phần mềm, các tính chất có thể thấy được từ bên ngoài của các thành phần này, và các liên kết giữa chúng”

- Các thành phần phần mềm có thể gồm:

- Các module
- Các cấu trúc dữ liệu, cơ sở dữ liệu



- Mục đích sử dụng:
 - Để đánh giá tính hiệu quả của phần mềm trong việc đáp ứng các y/c của hệ thống
 - Cân nhắc để chọn ra kiến trúc phù hợp nhất giữa các kiến trúc khác nhau
 - Giúp dự trù sớm và tương đối chính xác các tài nguyên cần chuẩn bị cho giai đoạn cài đặt phần mềm
 - Đóng vai trò như thiết kế tổng thể, làm nền tảng cho các thiết kế chi tiết sau đó
 - Giúp giảm thiểu các rủi ro trong quá trình xây dựng phần mềm sau này



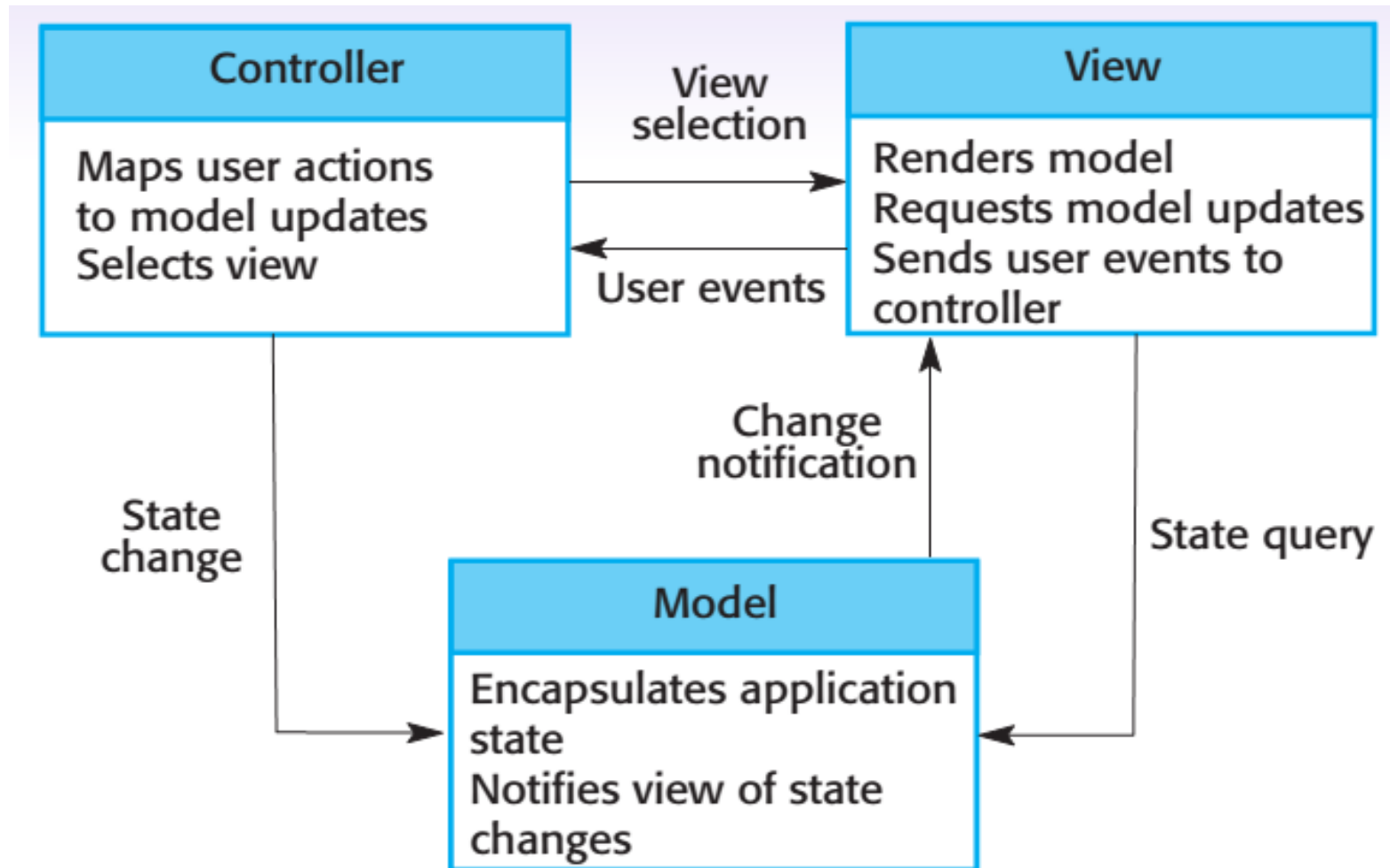
- Các mô hình kiến trúc
 - Mô hình Model-View-Controller (MVC)
 - Mô hình kiến trúc phân tầng
 - Mô hình Repository
 - Mô hình client–server
 - Mô hình pipe and filter



Mô hình Model-View-Controller (MVC)

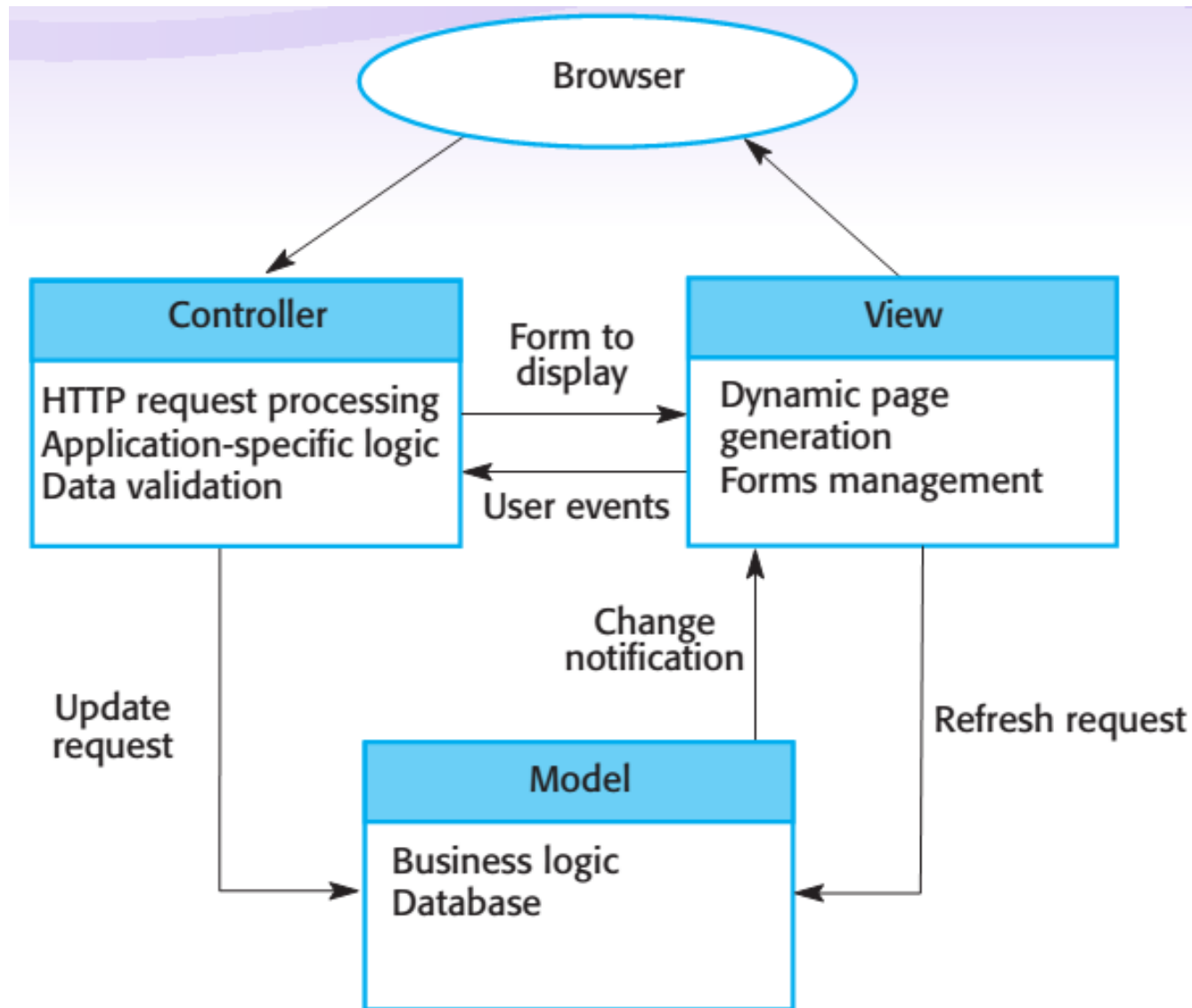
Tên	Mô hình MVC (Model-View-Controller)
Mô tả	Tách riêng phần biểu diễn và phần tương tác ra khỏi dữ liệu hệ thống. Hệ thống được cấu trúc hóa thành ba component logic tương tác với nhau. - Model component: quản lý dữ liệu hệ thống và các thao tác trên dữ liệu đó. - View component: định nghĩa và quản lý cách dữ liệu được biểu diễn tới người dùng như thế nào. - Controller component: Quản lý tương tác người dùng (ví dụ như ấn phím, nhấp chuột, ...) và chuyển các tương tác này tới View và Model.
Sử dụng khi nào	Được sử dụng khi có nhiều cách biểu diễn và tương tác với dữ liệu. Cũng được sử dụng khi chưa biết được các yêu cầu tương lai cho tương tác và biểu diễn dữ liệu.
Ưu điểm	Cho phép dữ liệu thay đổi độc lập với việc biểu diễn của nó và ngược lại. Hỗ trợ biểu diễn theo nhiều cách khác nhau trên cùng một dữ liệu.
Nhược điểm	Có thể chứa code bổ sung và code sẽ phức tạp hơn khi mô hình dữ liệu và mô hình tương tác đơn giản.

@ Mô hình Model-View-Controller (MVC)





Ví dụ: Kiến trúc ứng dụng Web sử dụng mô hình MVC





Mô hình kiến trúc phân tầng

- Được sử dụng để mô hình hóa giao diện của các hệ thống con.
- Tổ chức hệ thống thành một tập các tầng, mỗi tầng cung cấp một tập các dịch vụ.
- Hỗ trợ việc phát triển dần dần các hệ thống con trên các tầng khác nhau. Khi giao diện của tầng thay đổi, chỉ các tầng lân cận mới bị ảnh hưởng.



Mô hình kiến trúc phân tầng

Tên	Kiến trúc phân tầng
Mô tả	Tổ chức hệ thống thành các tầng, mỗi tầng chứa các chức năng liên quan đến nhau. Một tầng cung cấp các dịch vụ cho tầng trên của nó vì vậy các tầng thấp nhất biểu diễn các dịch vụ lõi được sử dụng trong toàn bộ hệ thống.
Sử dụng khi nào	Được sử dụng khi xây dựng các tính năng mới dựa trên những hệ thống có sẵn; khi việc phát triển được dàn trải trên nhiều nhóm khác nhau và mỗi nhóm chịu trách nhiệm về chức năng của một tầng; khi có một yêu cầu về bảo mật ở nhiều mức độ.
Ưu điểm	Cho phép thay thế các phần miễn là interface được duy trì. Các chức năng dư thừa (ví dụ như phân quyền) có thể được cung cấp ở mỗi tầng để tăng độ tin cậy của hệ thống.
Nhược điểm	Thực tế, cung cấp một sự phân chia rõ rệt giữa các tầng thường rất khó khăn và tầng cao hơn có thể tương tác trực tiếp với tầng thấp hơn hơn là thông qua một tầng bên dưới nó. Hiệu năng cũng có thể là một vấn đề vì nhiều mức diễn giải của một yêu cầu dịch vụ khi nó được thực hiện tại mỗi tầng.

@ Mô hình phân tầng tổng quát

User interface

User interface management
Authentication and authorization

Core business logic/application functionality
System utilities

System support (OS, database etc.)



Mô hình Repository

- Các hệ thống con phải trao đổi dữ liệu với nhau. Có hai khả năng:
 - Việc chia sẻ dữ liệu được thực hiện ở cơ sở dữ liệu trung tâm hay còn gọi là kho dữ liệu, kho này được truy cập bởi tất cả các hệ thống con.
 - Mỗi hệ thống con duy trì một cơ sở dữ liệu riêng và chuyển dữ liệu trực tiếp tới các hệ thống con khác.
- Khi có một lượng lớn dữ liệu cần chia sẻ, mô hình về chia sẻ là mô hình phổ biến nhất và là cơ chế chia sẻ dữ liệu hiệu quả nhất.

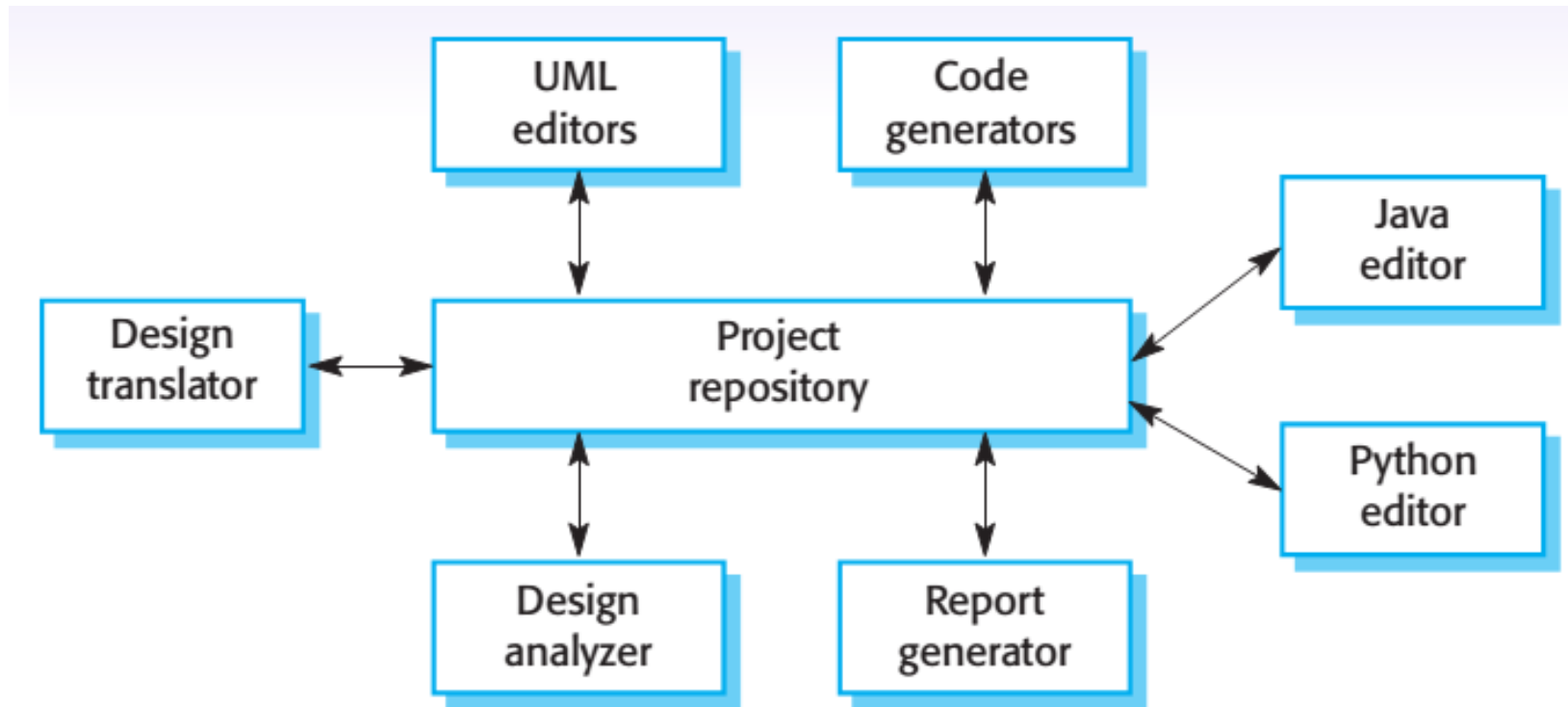


Mô hình Repository

Tên	Kiến trúc repository
Mô tả	Tất cả các dữ liệu trong hệ thống được quản lý ở một kho trung tâm, kho này được truy cập bởi tất cả các component của hệ thống. Các component không tương tác trực tiếp với nhau, chỉ thông qua kho chung thôi.
Sử dụng khi nào	Sử dụng mô hình này khi ta có một hệ thống trong đó một lượng lớn thông tin được sinh ra phải được lưu trữ trong một thời gian dài. Ta cũng có thể sử dụng nó trong các hệ thống hướng dữ liệu trong đó dữ liệu trong kho kích hoạt hành động hay công cụ.
Ưu điểm	Các component có thể độc lập với nhau – chúng không cần biết sự tồn tại của các component khác. Các thay đổi xảy ra ở một component không ảnh hưởng tới các component khác. Tất cả các dữ liệu có thể được quản lý một cách nhất quán (ví dụ như backup dữ liệu được thực hiện đồng thời) vì tất cả dữ liệu được lưu trữ ở cùng một nơi.
Nhược điểm	Các vấn đề xảy ra trên kho chung ảnh hưởng đến toàn hệ thống. Có thể không hiệu quả trong việc tổ chức các giao tiếp thông qua kho. Phân tán kho trên nhiều máy tính có thể khó khăn.



Một kiến trúc repository cho một IDE





Mô hình client - server

- Các mô hình hệ thống phân tán chỉ ra cách dữ liệu và các xử lý được phân tán trên nhiều component như thế nào.
- Tập hợp các server độc lập cung cấp các dịch vụ cụ thể ví dụ như in ấn, quản trị dữ liệu, ...
- Tập hợp các khách hàng triệu gọi các dịch vụ này.
- Hệ thống mạng cho phép người dùng truy cập vào các server.

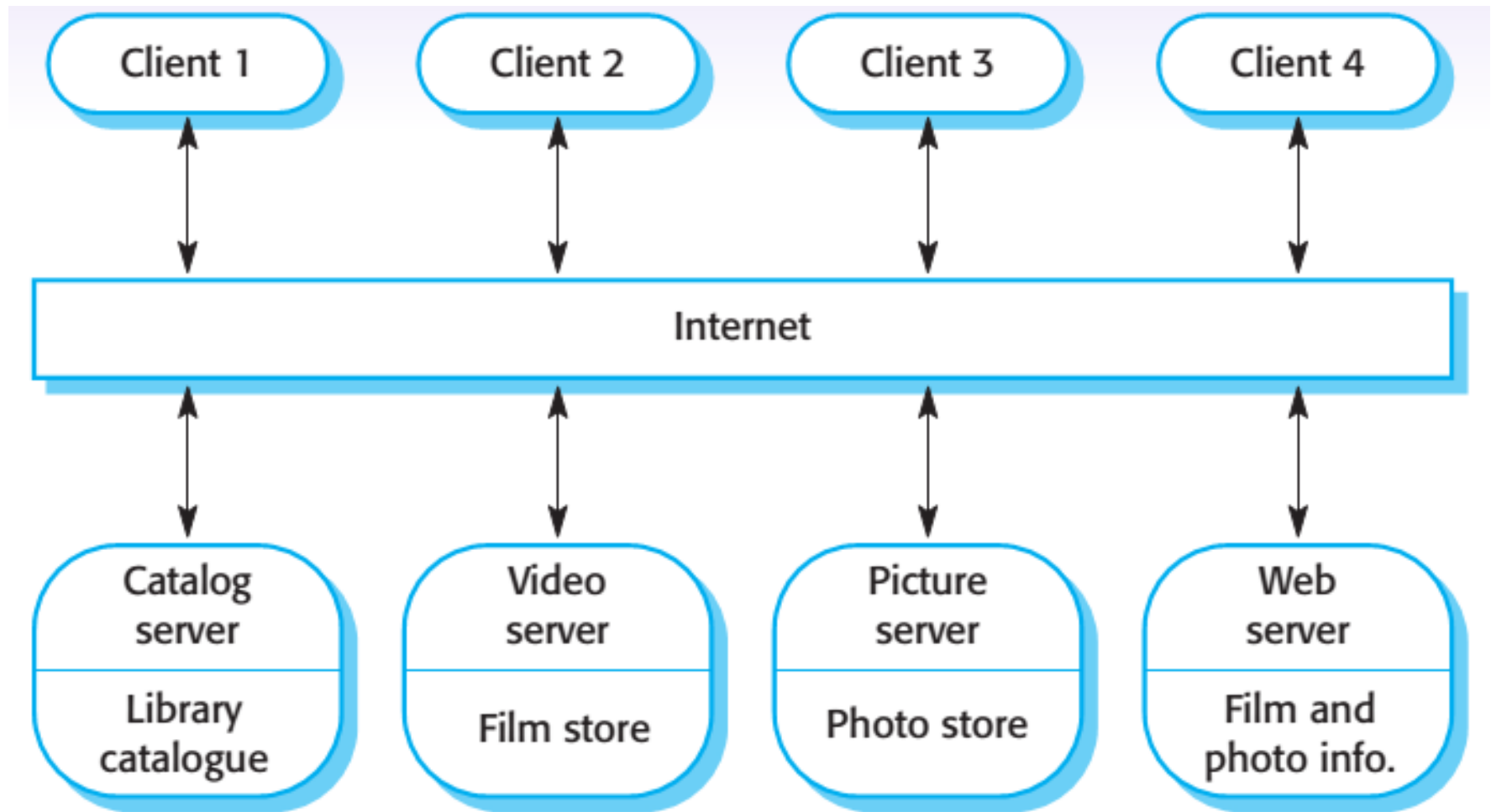


Mô hình client - server

Tên	Mô hình client - server
Mô tả	Trong một kiến trúc client-server, chức năng của hệ thống được tổ chức thành các dịch vụ, mỗi dịch vụ được đặt trên một server riêng lẻ. Khách hàng là người sử dụng các dịch vụ này và truy cập vào các server để sử dụng dịch vụ.
Sử dụng khi nào	Được sử dụng khi dữ liệu trong một cơ sở dữ liệu chia sẻ phải truy cập từ nhiều nơi. Vì các server được truy cập từ nhiều nơi khác nhau, có thể được sử dụng khi tải trên hệ thống thay đổi.
Ưu điểm	Ưu điểm chính là server được phân tán trên mạng. Chức năng chung (dịch vụ in ấn chẳng hạn) có thể có sẵn cho tất cả các khách hàng và không cần thiết phải cài đặt toàn bộ các dịch vụ.
Nhược điểm	Mỗi dịch vụ là một điểm đơn gây lỗi vì vậy dễ bị tấn công từ chối dịch vụ hoặc lỗi server. Hiệu năng có thể không dự đoán trước được do nó phụ thuộc vào mạng cũng như hệ thống. Có thể có các vấn đề về quản lý nếu server được sở hữu bởi các tổ chức khác nhau.

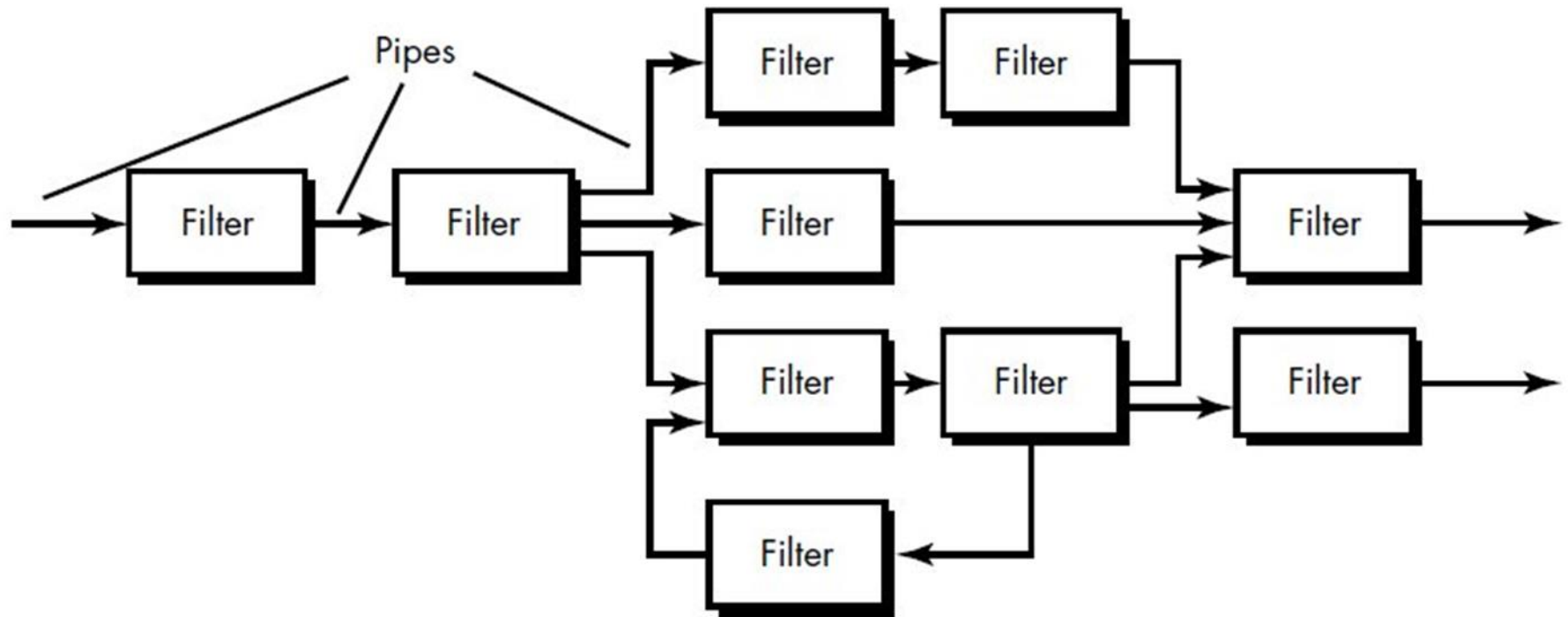


Ví dụ về mô hình client - server cho một thư viện phim ảnh



@ Mô hình pipe filter

- Các chuyển đổi chức năng xử lý các đầu vào và tạo ra các đầu ra.
- Các biến thể của phương pháp này rất phổ biến. Khi các chuyển đổi là tuần tự, đây là mô hình xử lý khối tuần tự mà các hệ thống xử lý dữ liệu sử dụng.
- Không thật sự phù hợp với các hệ thống tương tác.





@ Mô hình pipe filter

Tên	Mô hình pipe filter
Mô tả	Việc xử lý dữ liệu trong một hệ thống được tổ chức sao cho mỗi component xử lý (filter) là rời rạc và tiến hành một loại xử lý dữ liệu. Dòng dữ liệu (pipe) đi từ một component đến một component khác để xử lý.
Sử dụng khi nào	Thường sử dụng trong các ứng dụng xử lý dữ liệu (cả ứng dụng xử lý khối và xử lý giao tác) trong đó các đầu vào được xử lý ở các giai đoạn rời rạc để tạo ra các đầu ra tương ứng.
Ưu điểm	Dễ hiểu và hỗ trợ việc tái sử dụng chuyển đổi. Kiểu dòng dữ liệu phù hợp với cấu trúc của của nhiều quy trình công việc. Tiến hóa bằng cách thêm vào các chuyển đổi là dễ dàng. Có thể cài đặt theo kiểu hệ thống tuần tự hoặc song song.
Nhược điểm	Format của dữ liệu truyền đi phải được chấp thuận trong việc giao tiếp giữa các chuyển đổi. Mỗi chuyển đổi phải phân tích cú pháp đầu vào của nó và chuyển nó thành đầu ra ở dạng được chấp nhận. Điều này gây khó khăn trong việc tái sử dụng các chuyển đổi hàm mà cấu trúc dữ liệu không tương thích.

@ Ví dụ về mô hình pipe filter

